# WorksheetDownload.com

10 CONCEPTUAL Q&A  API TECHNOLOGIES · PRACTICE WORKSHEET

CONCEPTUAL SERIES
Azure API Manageme

Name: ________ Cohort/Batch: ________ Date: ________ Score: ________

## AZURE API MANAGEMENT PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A  API Technologies

TOTAL: 10 QUESTIONS

**Q1** What is Azure API Management and what API communication problem does it solve? *Threat Model*

**Q2** How does Azure API Management define its schema or contract? *Architecture*

**Q3** How does Azure API Management handle authentication? *Configuration*

**Q4** How does Azure API Management handle API versioning? *Hardening*

**Q5** How does Azure API Management handle errors and status codes? *SSO / IdP*

**Q6** How do you implement pagination in a Azure API Management API? *Observability*

**Q7** How do you document a Azure API Management API? *Lifecycle*

**Q8** How do you test a Azure API Management API? *Compliance*

**Q9** How do you protect a Azure API Management API from abuse? *Testing*

**Q10** What monitoring and observability should a Azure API Management API have? *Tuning*

# ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

### A1 Azure API Management defines a protocol — Mitigation

Azure API Management defines a protocol or standard for structured communication between services. It addresses concerns like schema definition, payload format, versioning, or transport efficiency that plain HTTP with ad-hoc JSON does not solve on its own.

### A6 Cursor-based pagination (next_cursor token) is — Observability

Cursor-based pagination (next_cursor token) is preferred for large or frequently-updated datasets because it is stable. Offset-based pagination (page, limit) is simpler but can skip or duplicate rows if data changes between requests.

### A2 Azure API Management uses an IDL — Primitives

Azure API Management uses an IDL or schema language to declare types, operations, and their inputs and outputs. This schema is the single source of truth from which documentation, validation, and client SDKs can be auto-generated.

### A7 Azure API Management APIs use an — Automation

Azure API Management APIs use an API specification (OpenAPI, AsyncAPI, GraphQL SDL) to generate interactive documentation (Swagger UI, ReDoc, GraphiQL). Good docs include examples, error codes, and authentication instructions.

### A3 Azure API Management APIs commonly use — Hardening

Azure API Management APIs commonly use Bearer tokens (JWT/OAuth2) in the Authorization header or API keys in custom headers. Transport-level security (TLS) is always required. Additional mechanisms (mTLS, HMAC) apply to high-trust scenarios.

### A8 Contract testing (Pact) verifies the provider — Governance

Contract testing (Pact) verifies the provider matches the consumer's schema. Integration tests call real endpoints against a test environment. Load tests (k6, Gatling) verify performance under production-like concurrency.

### A4 Common versioning strategies include URL path — Gotchas

Common versioning strategies include URL path (/v1/users), request headers (Accept: application/vnd.api+v2+json), and query params (?version=2). Azure API Management prefers one strategy and maintains backward compatibility within a major version.

### A9 Rate limiting (tokens per IP per — Assessment

Rate limiting (tokens per IP per minute) prevents DoS. Input validation rejects malformed requests before they reach business logic. Output filtering (response schema) prevents accidental data leakage. Monitor for anomalous usage patterns.

### A5 Azure API Management uses HTTP status — Federation

Azure API Management uses HTTP status codes (200 OK, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error) combined with a structured error body containing a code, message, and optional details field.

### A10 Instrument request count, latency histograms, — Optimization

Instrument request count, latency histograms, and error rate with Prometheus or a cloud metrics system. Add distributed tracing with OpenTelemetry. Log structured request/response data. Set SLOs and alert when SLIs breach them.